UNIVERSIDAD DE LA EMPRESA
Licenciatura en Informática
Curso: Taller II – 2026

Análisis y diseño

Entrega final

Desarrollo de sistema para emprendimiento familia

Integrantes del grupo:
Shoan Genes - 55932232
Valdo Castiglia - 56567977
Mateo Buschiazzo - 54752603

Fecha de entrega: 10/3/2026
Docente: José Clavijo

Introducción

El presente informe tiene como objetivo describir el análisis, diseño e implementación de un primer prototipo de un sistema de gestión de ventas y reparto a domicilio para un emprendimiento familiar dedicado a la elaboración de postres artesanales, en el marco del curso Taller II de la Licenciatura en Informática de la Universidad de la Empresa. El trabajo consiste en desarrollar una solución orientada a objetos que permita registrar y administrar de forma eficiente los postres ofrecidos, las ventas realizadas y los repartos asociados, aplicando conceptos fundamentales como encapsulamiento, herencia, polimorfismo, estructuras de datos dinámicas y concurrencia.

A partir de la especificación proporcionada en la letra del trabajo obligatorio, se elaboró un modelo conceptual que sirvió como base para las decisiones de diseño adoptadas. Dichas decisiones incluyen la definición de las clases que conforman el dominio del problema, la organización de las colecciones internas, la elección de estructuras de datos adecuadas y la justificación técnica de cada una de ellas. Asimismo, se buscó garantizar un diseño claro y coherente, evitando la duplicación innecesaria de información y facilitando la extensibilidad del sistema.

El sistema fue diseñado siguiendo una arquitectura cliente–servidor, donde los vendedores interactúan con la aplicación a través de un cliente con interfaz gráfica, mientras que la lógica de negocio y la persistencia de los datos se concentran en un servidor central. La comunicación entre cliente y servidor se realiza mediante Remote Method Invocation (RMI), permitiendo la invocación de métodos remotos de forma transparente y segura.

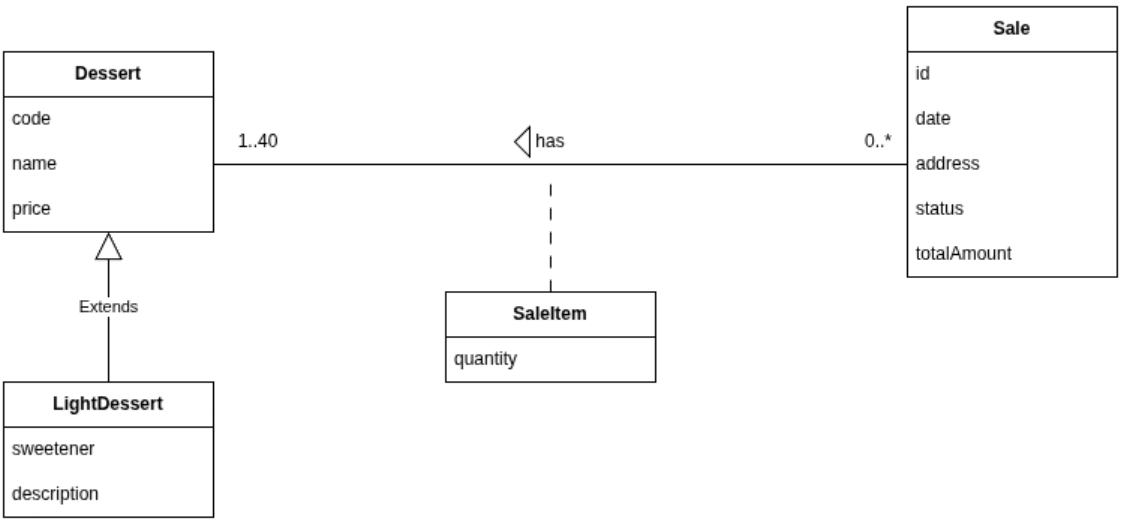
Finalmente, el informe presenta el desglose de los requerimientos funcionales solicitados, así como el modelo de clases de implementación que sirvió de guía para el desarrollo del sistema. De esta manera, se pretende ofrecer una visión general del funcionamiento de la aplicación y dejar constancia de las principales decisiones técnicas adoptadas y su correspondencia con los requerimientos planteados.

Diagrama de clases conceptual

El diagrama conceptual presentado modela las principales entidades del sistema y las relaciones entre ellas, permitiendo representar de forma clara la estructura y el comportamiento esperado de la aplicación. La entidad Dessert representa los postres ofrecidos por el sistema, identificados de forma única mediante un código, y contiene la información común a todos los tipos de postres, como nombre y precio. A partir de esta clase se deriva LightDessert, que extiende el concepto de postre incorporando atributos específicos como el tipo de endulzante y una descripción adicional, aplicando el principio de herencia para evitar duplicación de información.

La entidad Sale representa una venta realizada, almacenando datos relevantes como número de venta, fecha, dirección de entrega, estado y monto total. Una venta puede contener entre 1 y 40 postres, restricción que se refleja explícitamente en el diagrama mediante la cardinalidad de la asociación correspondiente.

Para modelar correctamente la relación entre ventas y postres, se introduce la clase intermedia SaleItem. Esta clase permite representar una relación muchos-a-muchos entre Sale y Dessert, incorporando además el atributo quantity, que indica la cantidad de unidades de un determinado postre dentro de una venta. El uso de una clase de asociación evita la duplicación de información del postre y permite mantener una referencia a los postres existentes en el sistema, logrando un diseño más flexible, normalizado y fácil de mantener.



- [Link del diagrama aquí]

Elección de Diseño

A continuación se presenta la elección de diseño adoptada para el sistema, expresada a través de las estructuras de datos y clases definidas a partir del modelo conceptual y los requerimientos planteados.

Desserts = Diccionario(Dessert)
Sales = Secuencia(Sale)
Dessert = code x name x price
LightDessert = Dessert + (sweetener x description)
SaleItem = dessert x quantity
Sale = id x date x address x status x totalAmount x Secuencia(SaleItem)

Observaciones:

- Existe una única colección global para las clases Sale y Dessert. A diferencia de estas, cada instancia de Sale posee su propia secuencia de SaleItem.
- Se utiliza un diccionario para la colección Desserts, ya que cada postre posee identificador único (códigos alfanuméricos).
- Se utiliza una secuencia para la colección ventas, ya que se desea almacenar las ventas de forma secuencial a medida que se van realizando.
- Cada SaleItem mantiene una referencia a un Dessert existente en la colección Desserts, evitando la duplicación de información.
- En toda instancia de Sale, la suma de las cantidades de los SaleItem no puede superar las 40 unidades.

Desglose de requerimientos

Requerimiento: Registrar los datos de un nuevo postre que se pone a la venta en la tiendita, verificando que no haya otro postre previamente registrado con el mismo código.	
Entrada: Datos del nuevo postre.	
Salida: Postre ingresado en el sistema.	

Proceso	Verificar que no pertenezca otro postre con el mismo código en el diccionario Desserts
	si pertenece **entonces**
	 [ERROR]: Ya existe un postre con ese identificador en el sistema
	sino
	 Verificar que el nombre del postre no sea vacío
	 si es vacío **entonces**
	  [ERROR]: El nombre de un postre no puede ser vacío
	 sino
	  Verificar que el precio del postre sea positivo
	  si precio < 0 **entonces**
	   [ERROR]: El precio de un postre debe de ser positivo
	  sino
	   Ingresar el postre en el sistema
	  fin si
	 fin si
	fin si
	fin

Requerimiento: Obtener un listado conteniendo código, nombre, precio unitario y tipo (común o light) de todos los postres registrados en el sistema. Este listado se realizará ordenado alfanuméricamente por código de postre.	
Entrada: Diccionario de postres	
Salida: Listado de los postres ordenados por código	
Proceso	Recorrer el diccionario Desserts **para cada** postre **hacer** Insertar el postre en una lista **fin para** **retornar** la lista conteniendo a todos los postres del diccionario **fin**

Requerimiento: Dado el código que identifica a un postre, listar todos sus datos (código, nombre, precio unitario y tipo). En caso de tratarse de un postre light, se listará también el endulzante empleado junto con su descripción adicional.	
Entrada: Código identificador de un postre.	
Salida: Los datos del postre correspondiente a dicho identificador.	
Proceso	Verificar que exista un postre con dicho identificador en el diccionario Desserts **si** no existe **entonces** **[ERROR]:** No existe un postre con el identificador seleccionado **sino** **retornar** los datos del postre seleccionado **fin si** **fin**

Requerimiento: Registrar los datos básicos (fecha y dirección de entrega) de una nueva venta que comienza, la cual se marcará automáticamente como venta en proceso y su monto total será inicialmente cero. Dado que los números de venta son secuenciales, se le asignará en forma automática el número siguiente al de la última venta registrada hasta el momento, verificando además que la fecha de la nueva venta sea igual o posterior a la misma	
Entrada: Fecha y dirección de entrega	
Salida: Venta registrada en el sistema	

Proceso	Verificar que la dirección no sea vacía
	si es vacía **entonces**
	 [ERROR]: La dirección ingresada no puede ser vacía
	sino
	 Verificar que la fecha sea igual o posterior a la de la última venta
	 si fecha > última fecha **entonces**
	  [ERROR]: La fecha de una venta debe ser igual o posterior a la fecha de la última fecha realizada
	 sino
	  Registrar venta en el sistema, inicializando el identificador con el siguiente al de la última venta realizada y el status en IN_PROGRESS, para posteriormente insertar la venta al final de la secuencia Sales
	 fin si
	fin si
	fin

Requerimiento: Dados el código que identifica a un postre, una cantidad de unidades y el número que identifica a una venta en proceso, agregar dicha cantidad de unidades del postre a esa venta. Si el postre ya había sido agregado en forma previa, simplemente se incrementará la cantidad de unidades que se están adquiriendo. En cualquier caso, no se deben superar las 40 unidades de postres en total para la venta.

Entrada: Código identificador de un postre, cantidad de unidades y el número de venta.

Salida: Los datos de la venta se ven actualizados.

Proceso

```
Verificar que exista un postre con dicho identificador en el diccionario Desserts
si no existe entonces
    [ERROR]: No existe un postre con el identificador seleccionado
sino
    Verificar que exista una venta con dicho número en la secuencia Sales
    si no existe entonces
        [ERROR]: No existe ningún proceso de venta con el número
        proporcionado
    sino
        Verificar que las unidades sean positivas
        si las unidades son negativas entonces
            [ ERROR ]: Las unidades a incrementar deben ser positivas
        sino
            Verificar que el estado de la venta sea IN_PROGRESS
            si status != IN_PROGRESS entonces
                [ERROR]: El proceso de venta ya ha sido completado. No
                puede ser modificado
            sino
                Verificar que la suma de las unidades actuales y las nuevas
                no supere la cantidad de 40 unidades
                si unidades > 40 entonces
                    [ERROR]: Una venta no puede superar las 40
                    unidades
                sino
                    Si el postre ya estaba incluido en la venta,
                    incrementar sus unidades, sino añadirlo a la venta,
                    con las cantidades proporcionadas
                fin si
            fin si
        fin si
    fin si
fin si
fin
```

Requerimiento: Dados el código que identifica a un postre, la cantidad de unidades a eliminar, y el número que identifica a una venta en proceso, eliminar dicha cantidad de unidades de ese postre de la venta. Si se eliminan todas las unidades de ese postre, también se procederá a quitar totalmente ese postre de la venta.

Entrada: Código de postre, cantidad a eliminar, número de venta.

Salida: Postre eliminado de la venta en el sistema.

Proceso

```
Verificar que exista un postre con el identificador en el diccionario de Desserts
si no existe entonces
    [ERROR]: No existe el Postre con el identificador seleccionado
sino
    Verificar que exista una venta con ese número en la secuencia Sales
    si no existe entonces
        [ERROR]: No existe una venta con el número proporcionado
    sino
        Verificamos que la cantidad sea mayor que 0
        si cantidad <= 0 entonces
            [ERROR]: La cantidad a eliminar es inválida
        sino
            Verificar que el estado de la venta sea IN_PROGRESS
            si status != IN_PROGRESS entonces
                [ERROR]: El proceso de venta ya ha sido completado. No
                puede ser modificado
            sino
                Verificar que en la secuencia de SaleItems exista dicho postre
                si no existe entonces
                    [ERROR]: No se ha ordenado dicho postre
                sino
                    Obtenemos SaleItem que corresponda
                    Verificamos que cantidad <= cantidadActual
                    si cantidad > cantidadActual entonces
                        [ERROR]: No existen suficientes unidades para
                        eliminar
                    sino
                        Restamos cantidad al SaleItem
                    sin si
                fin si
            fin si
        fin si
    fin si
fin si
fin
```

Requerimiento: Dados el número que identifica a una venta en proceso y una indicación de si se confirma o se cancela, proceder a su confirmación o cancelación. Si se confirma, se marcará como finalizada, continuará almacenada en el sistema y se emitirá al usuario el monto total a abonar. Si se cancela, será eliminada en ese momento. También será eliminada si no tiene registrado ningún postre, sin importar si se confirma o cancela.	
Entrada: Número identificador de una venta.	
Salida: Venta confirmada o cancelada.	

Proceso	Verificar si existe una venta con el número proporcionado en la secuencia de ventas Sales
	si no existe **entonces**
	  [ERROR]: No existe un proceso de venta con el identificador proporcionado
	sino
	  Verificar que el status de la venta no sea COMPLETED
	  si status == COMPLETED **entonces**
	  [ERROR]: La venta ya ha sido completada con anterioridad
	  sino
	  Verificar si la cantidad de ítems de la venta es cero
	  si cantidad de ítems == 0 **entonces**
	  Borrar la venta del sistema
	  sino
	  según
	  caso COMPLETE **entonces**
	  Setear el status como COMPLETED
	  caso CANCEL **entonces**
	  Borrar la venta del sistema
	  fin según
	  fin si
	  fin si
	fin si
	fin

Requerimiento: Dada una indicación (que puede ser T, P ó F) obtener un listado de las ventas correspondientes a dicha indicación (T = todas las ventas, P = ventas en proceso, F = ventas finalizadas). Por cada venta que integre el listado se desplegará número, fecha, dirección de entrega, monto total de la venta y estado (en proceso o finalizada), ordenado en forma ascendente por número de venta.	
Entrada: Una indicación para filtrar las ventas por status.	
Salida: Listado de todas las ventas que cumplan con la indicación.	
Proceso	**si** indicador == T **entonces** Mapear la secuencia Sales a una lista **retornar** dicha lista **sino** SaleStatus filter = indicador == P ? IN_PROGRESS : COMPLETED **para cada** Sale **hacer** Insertar todas las ventas cuyo estatus sea igual a filter en una lista **fin para** **retornar** la lista que contenga todas las ventas requeridas **fin si** **fin**

Requerimiento: Dado el número que identifica a una venta, obtener un listado de todos los postres agregados a dicha venta (código, nombre, precio unitario, tipo de postre y cantidad de unidades). Los postres serán listados en el mismo orden en que fueron siendo agregados a la venta.	
Entrada: Número identificador de una venta.	
Salida: Listado de postres de la venta.	
Proceso	Verificar si existe una venta con el número proporcionado en la secuencia Sales **si** no existe **entonces** **[ERROR]:** No existe un proceso de venta con el identificador proporcionado **sino** Obtenemos la venta Recorremos la Secuencia de SaleItem **para cada** SaleItem **hacer** Insertar cada uno en una lista **fin para** **retornar** la lista conteniendo todos postres de la venta **fin si** **fin**

Requerimiento: Dados el código de un postre y una fecha, calcular el monto total de dinero recaudado por concepto de ventas (finalizadas), junto con la cantidad total de unidades vendidas de dicho postre en esa fecha. Si no se vendió ninguna unidad de dicho postre en esa fecha, ambos resultados serán cero.

Entrada: Código de un postre y una fecha.

Salida: Monto total de dinero y cantidad de unidades vendidas de dicho postre en esa fecha.

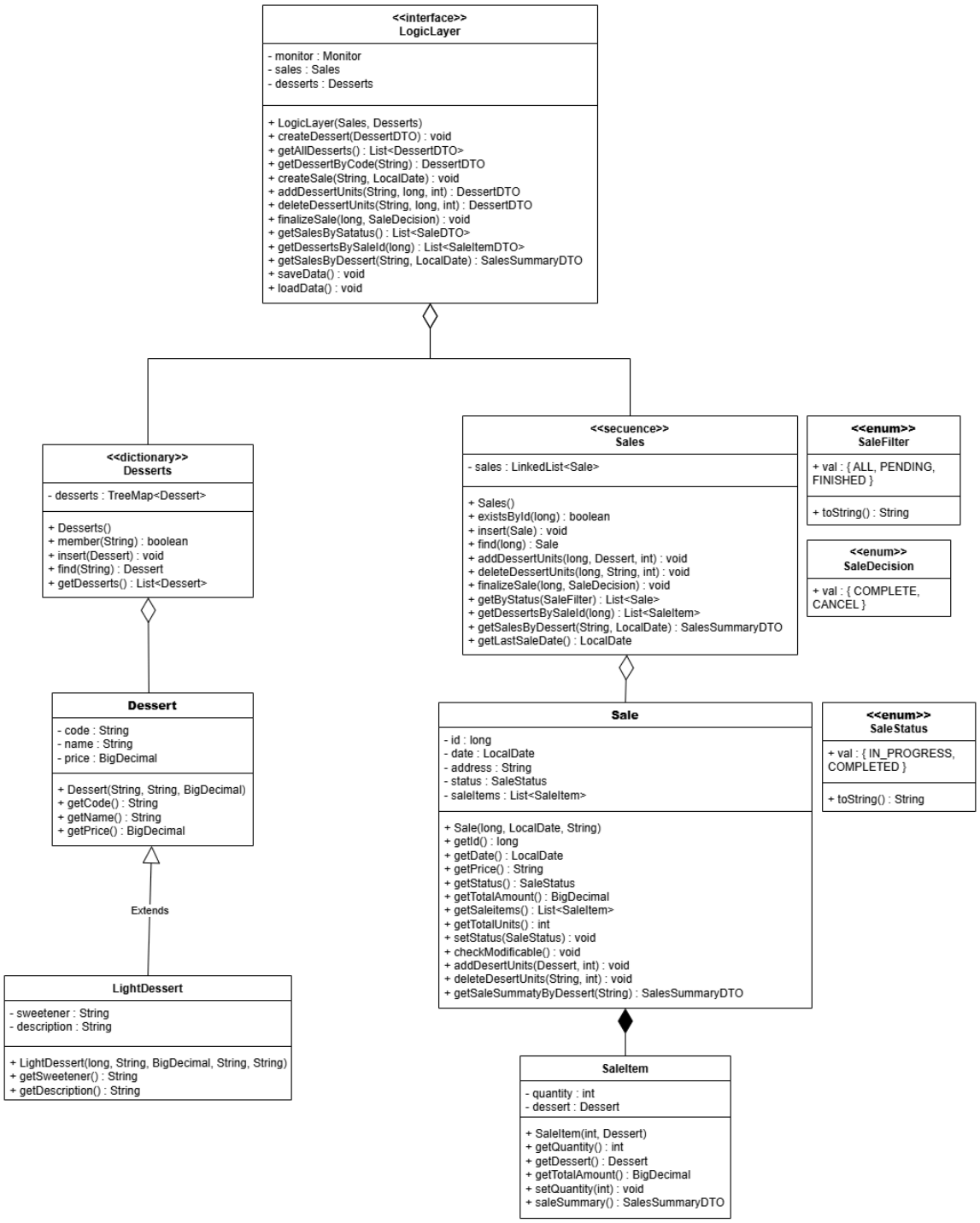
Proceso

```
Verificar que el código sea válido
si es vacio entonces
    [ERROR]: Código invalido
sino
    Verificar que el nombre del postre exista
    si no existe entonces
        [ERROR]: El postre ingresado no existe
    sino
        totalMonto = 0
        totalUnidades = 0
        Recorremos la Secuencia de sales
        para cada Sale hacer
            Obtener el status
            si status == COMPLETED entonces
                Obtener la fecha
                si fechaDada == fecha de la venta entonces
                    i = 0
                    encontrado = false
                    mientras i < cantidad de saleItems  y no encontrado
                    hacer
                        si saleItems[i].dessert.code == código buscado
                        entonces
                            totalUnidades += quantity del SaleItem
                            totalMonto += quantity * price (precio unitario)
                            encontrado = true
                        fin si
                        i++
                    fin mientras
                fin si
            fin si
        fin para
    fin si
Encapsular resultados en un Data Transfer Object
retornar el data transfer object
fin
```

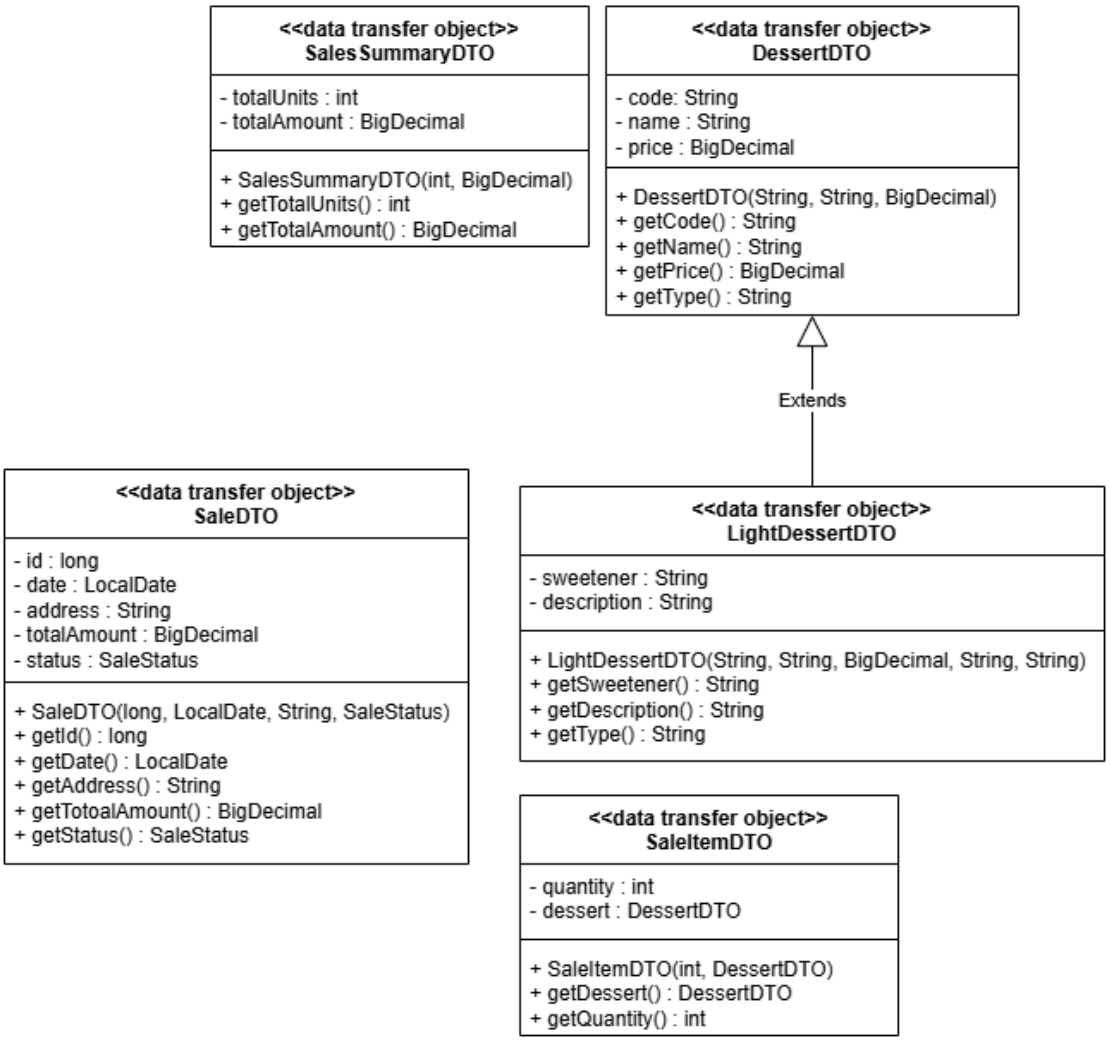
Requerimiento: Respaldar en disco todos los datos de la aplicación. Este requerimiento podrá ejecutarse toda vez que algún vendedor lo considere oportuno, especialmente luego de haber ejecutado funcionalidades que produzcan cambios en la información del sistema. Todos los datos se respaldarán juntos en un único archivo binario en disco (ubicado en el servidor central), para luego poder ser recuperados a memoria en una próxima ejecución.	
Entrada: Datos de la aplicación	
Salida: Todos los datos de la aplicación respaldados en un archivo	
Proceso	Reunir todos los datos del programa Verificar si existe un archivo un archivo de respaldo **si** no existe **entonces** Se crea un archivo binario **fin si** Realizamos el respaldo de los datos en el archivo **fin**

Requerimiento: Recuperar a memoria todos los datos de la aplicación almacenados en disco. Este requerimiento será ejecutado automáticamente en el servidor central cada vez que el sistema inicie su ejecución (no será disparado por ningún usuario que explícitamente solicite su ejecución).	
Entrada: Nombre del archivo	
Salida: Todos los datos de la aplicación cargados en memoria	
Proceso	Verificar que existe el archivo de respaldo **si** no existe **entonces** Inicializar las estructuras de datos vacías **sino** Abrir el archivo y leer los datos **fin si** **fin**

Diagrama de Implementación



- [Link del diagrama aquí]



- [Link del diagrama aquí]

Estructuras de datos

En esta sección se describen y justifican las estructuras de datos seleccionadas para implementar las distintas colecciones del sistema. La elección de cada estructura se realiza en función de las características del problema, los requerimientos funcionales planteados y las operaciones que se ejecutan con mayor frecuencia sobre los datos.

Se prioriza el uso de estructuras dinámicas, dado que la cantidad de elementos a manejar no se conoce de antemano y puede variar a lo largo del tiempo. Asimismo, se busca mantener la coherencia con el modelo orientado a objetos definido, evitando la duplicación de información y favoreciendo un diseño claro, flexible y extensible.

A continuación, se detalla la justificación de las estructuras utilizadas para cada una de las colecciones principales del sistema.

Diccionario Desserts

En el caso de la colección de postres, la elección de la estructura de datos se desprende directamente de las características del problema planteado. El sistema debe manejar una cantidad indeterminada de postres, la cual puede crecer o disminuir dinámicamente a lo largo del tiempo, lo que descarta el uso de estructuras estáticas y hace necesaria la utilización de memoria dinámica.

Cada postre se identifica de manera única mediante un código alfanumérico, el cual se utiliza como clave para su almacenamiento y recuperación. Dado que el sistema requiere acceder frecuentemente a los postres a partir de dicho código, resulta apropiado el uso de una estructura basada en pares clave – valor, que permita búsquedas directas sin necesidad de recorrer la colección completa.

Por este motivo, la colección Desserts se implementa como un diccionario, utilizando una estructura de tipo árbol (TreeMap), que Java implementa internamente como un árbol de búsqueda balanceado. Esta elección permite almacenar los postres de forma ordenada según su código, facilitando la obtención de listados ordenados sin necesidad de realizar operaciones adicionales de ordenamiento, a su vez que optimiza las búsquedas por identificador a O(log(n)).

Asimismo, dado que existen distintos tipos de postres (normales y light), y que estos se modelan mediante herencia a partir de la clase Dessert, la colección almacena referencias al tipo base. Esto permite aprovechar el polimorfismo, evitando duplicación de estructuras y garantizando la correcta invocación de los métodos correspondientes según el tipo concreto de postre.

Secuencia Sales

La colección de ventas debe almacenar una cantidad indeterminada de elementos, dado que el número de ventas realizadas por la tiendita puede crecer de forma sostenida a lo largo del tiempo. Esta característica descarta el uso de estructuras estáticas y requiere el empleo de una estructura de datos dinámica.

Las ventas se identifican mediante un número entero incremental, asignado de forma secuencial, y se almacenan respetando dicho orden. En función de esto, la colección Sales se implementa como una secuencia utilizando una lista enlazada (LinkedList), lo que permite mantener el orden natural de las ventas según su número sin necesidad de realizar operaciones adicionales de ordenamiento.

Dado que Java internamente implementa está colección como una lista doblemente encadenada con puntero al principio y al final, las operaciones de inserción al final están optimizadas a O(1).

La elección de una lista enlazada resulta adecuada dado que las operaciones más frecuentes sobre la colección son la inserción de nuevas ventas y el recorrido secuencial de las mismas para su listado. Asimismo, la estructura permite la eliminación de ventas de forma eficiente, operación requerida cuando una venta en proceso es cancelada.

Secuencia SaleItem

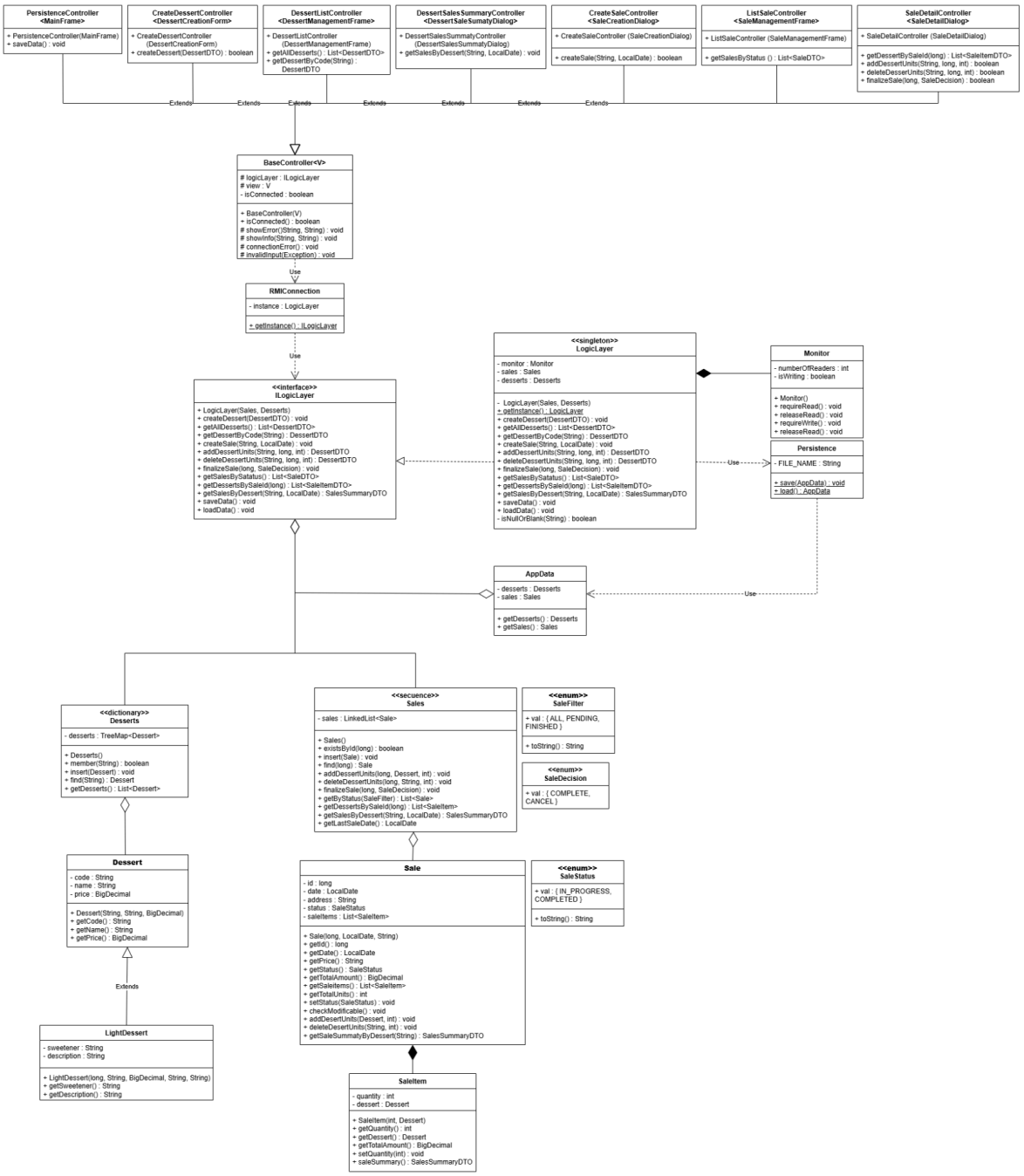
Cada venta puede estar compuesta por múltiples postres, y para cada uno de ellos es necesario registrar la cantidad de unidades adquiridas. Para modelar esta relación se introduce la clase SaleItem, que actúa como clase de asociación entre Sale y Dessert, permitiendo agrupar un postre con la cantidad correspondiente dentro de una venta.

Los SaleItem asociados a una venta se almacenan en una secuencia, implementada mediante una lista. Esta elección permite manejar dinámicamente la cantidad de ítems que componen cada venta, sin imponer una estructura rígida basada en arreglos con tope fijo.

Si bien el enunciado establece que una venta no puede superar las 40 unidades en total, dicha restricción se interpreta como una condición lógica del dominio y no como una limitación estructural. Por este motivo, el control del máximo de unidades se realiza mediante una validación sobre la secuencia de SaleItem, evitando acoplar el modelo de datos a una estructura estática que limitaría la extensibilidad y claridad del diseño. Por lo tanto la estructura a utilizar para la implementación es una LinkedList, una implementación de una lista doblemente encadenada con puntero al principio y al final.

La utilización de una secuencia de SaleItem favorece un diseño orientado a objetos más flexible, evita la duplicación de referencias a postres y mantiene separadas las responsabilidades de modelado y validación de las reglas del negocio.

Diagrama Completo



- [Link del Diagrama aquí]

Conclusión

El desarrollo del sistema permitió aplicar en la práctica conceptos fundamentales de la programación orientada a objetos y los sistemas distribuidos. La separación en capas (presentación, lógica y datos) facilitó el desarrollo independiente de cada componente y permitió que los cambios en una capa no afectaran a las demás.

La implementación de RMI como mecanismo de comunicación remota resultó transparente para la capa de presentación, que interactúa con la lógica del servidor a través de la interfaz ILogicLayer sin conocer los detalles de la comunicación de red. El uso del patrón Singleton tanto en LogicLayer como en RMIConnection garantizó una única instancia del servidor y una única conexión por cliente.

El control de concurrencia mediante el Monitor implementado permitió gestionar el acceso simultáneo de múltiples clientes, asegurando consistencia en las operaciones de lectura y escritura sobre las colecciones compartidas. El uso de finally en cada operación garantiza que los locks siempre se liberen incluso ante excepciones.

La persistencia mediante serialización Java resultó una solución simple y efectiva para el alcance del proyecto, permitiendo guardar y restaurar el estado completo de la aplicación entre sesiones.